Capítulo 15. El ingeniero de software impulsado por IA

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *No es la especie más fuerte la que sobrevive, ni la más inteligente, sino la que mejor se adapta al cambio.*
>
> —Charles Darwin

En 1801, Joseph Marie Jacquard inventó un telar controlado por tarjetas perforadas que podía tejer patrones intrincados automáticamente. Los tejedores profesionales observaban con inquietud cómo esta nueva máquina replicaba un trabajo que antes requería años de práctica. Muchos predijeron que esto supondría el fin de su oficio tal y como lo conocían.

Sin embargo, ocurrió algo inesperado. En lugar de eliminar a los tejedores, el telar Jacquard los transformó. Los artesanos expertos se convirtieron en diseñadores de patrones, operadores de máquinas e inge-

nieros textiles. Los tejedores más exitosos fueron aquellos que comprendieron tanto el oficio tradicional como la nueva tecnología. La producción se disparó, creando roles completamente nuevos que no existían antes.

Doscientos años después, nos encontramos en una situación similar. La IA escribe código, corrige errores y diseña sistemas completos. Los desarrolladores tienen los mismos pensamientos que tenían los tejedores en aquel entonces: «¿Estoy a punto de ser sustituido?». Pero aquí está la cuestión: la historia nos dice que normalmente no suele ser así. Este patrón se ha repetido con cada gran avance tecnológico y, en cada ocasión, las personas que se adaptaron salieron ganando.

El telar Jacquard no dejó sin trabajo a los tejedores cualificados. En cambio, les dio un superpoder: la capacidad de producir mucho más en el mismo tiempo. Un tejedor que antes tardaba días en crear un patrón complejo ahora podía producir varias piezas en un solo día. El mismo cambio fundamental está ocurriendo con la inteligencia artificial (IA) para los desarrolladores. Ahora puedes crear aplicaciones, depurar código y resolver problemas mucho más rápido que nunca.

Entonces, ¿en qué situación se encuentran hoy en día los desarrolladores como tú? Si aprendes a dominar los fundamentos de la ingeniería de software y a aprovechar la IA como tu programador en pareja, serás muy solicitado, no obsoleto. Tu capacidad para aprender, resolver problemas y adaptarte son tus mayores fortalezas, y mientras sigas desarrollando estas habilidades, seguirás siendo relevante. En la siguiente sección, aprenderás qué es la IA desglosando algunos de los conceptos con los que te encontrarás como desarrollador.

¿Qué es realmente la IA?

Aunque la IA parece haberse colado en todas las conversaciones que mantenemos hoy en día, no es algo nuevo. Los informáticos llevan persiguiendo el sueño de la IA desde la década de 1950, cuando los ordenadores ocupaban habitaciones enteras y tenían menos potencia de procesamiento que el teléfono que llevas en el bolsillo. La investigación en IA ha avanzado en oleadas de entusiasmo seguidas de «inviernos de la IA», cuando se agotaba la financiación para la IA de uso general. Sin embargo, la IA práctica nunca desapareció. Entre bastidores, la IA impulsaba silenciosamente cosas como la detección de fraudes, el reconocimiento de imágenes y los motores de búsqueda. Luego, en noviembre de 2022, OpenAI lanzó ChatGPT al público. En cinco días, tenía un millón de usuarios. En dos meses, 100 millones. De repente, la IA no era solo una curiosidad de investigación, sino una herramienta que cualquiera podía aprovechar.

Cuando se trata de responder a la pregunta «¿Qué es la IA?», no hay una respuesta sencilla, ya que se trata de una disciplina muy amplia. *La IA* es un término genérico que abarca desde simples sistemas basados en reglas hasta redes neuronales que intentan imitar la forma en que el cerebro humano procesa la información. Para que esto resulte relevante para los desarrolladores de software, nos centraremos en un conjunto específico de técnicas que han demostrado su eficacia para resolver algunos problemas muy interesantes.

En esta sección, haremos que la IA sea más accesible desmitificando algunos de los términos clave que pueden parecer intimidantes. Apren-

derás en qué aspectos destaca la IA y en cuáles se queda corta. Por último, exploraremos lo que la IA puede hacer específicamente por ti como ingeniero de software.

Desmitificando la terminología de la IA

Aprender una nueva tecnología a menudo puede resultar abrumador, especialmente cuando se trata de dominar toda la terminología que se puede encontrar en un nuevo ámbito. En IA, te encontrarás con términos como aprendizaje automático (ML), Deep Learning, IA generativa (GenAI) y LLMs, que se muestran en la figura 15-1. En esencia, la IA moderna enseña a los ordenadores a reconocer patrones o probabilidades y a seleccionar la respuesta más probable desde el punto de vista estadístico, a diferencia de la programación tradicional, en la que se escriben instrucciones paso a paso.

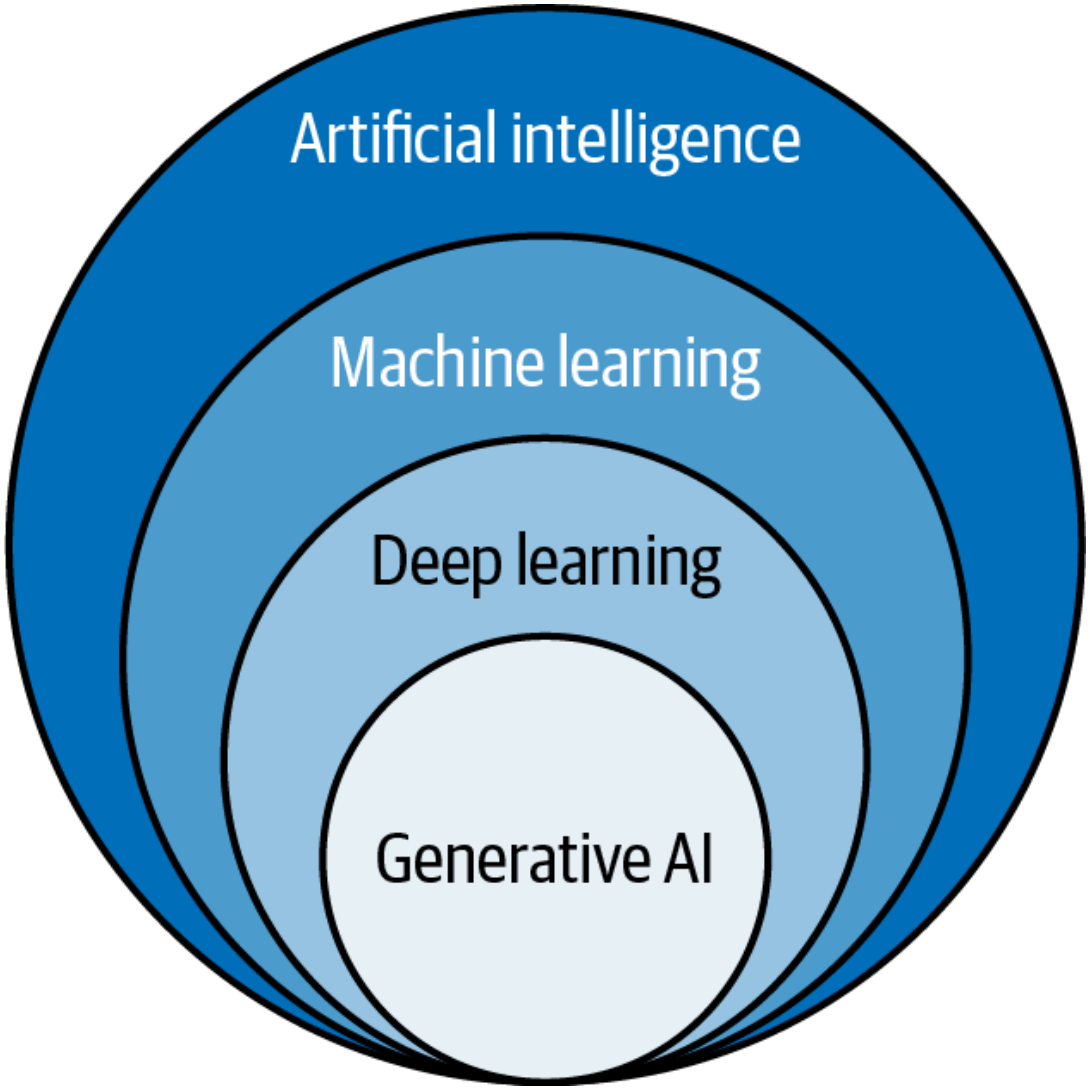


Figura 15-1. Las relaciones entre la IA, el aprendizaje automático, Deep Learning y la IA generativa

Como ingeniero de software, no es necesario que te conviertas en investigador de IA, pero comprender estos conceptos básicos te ayudará a integrar las capacidades de IA en tus aplicaciones y a comunicarte de manera eficaz con los científicos de datos y los ingenieros de IA de tu equipo.

Aprendizaje automático

El aprendizaje automático (*ML*) es la base de la IA moderna. En lugar de programar explícitamente cada decisión, los algoritmos de ML aprenden patrones a partir de los datos. El ejemplo clásico es el software de reconocimiento facial. No escribes código para describir cada rostro posible. En su lugar, alimentas al sistema con miles de imágenes de rostros y este aprende a identificar los patrones que hacen que cada rostro sea único.

En la programación tradicional, se sigue un modelo de «entrada + programa = salida» escribiendo reglas explícitas. En el ML, se invierte este modelo a «entrada + salida = programa» proporcionando ejemplos de entradas y sus salidas correctas. A continuación, los algoritmos determinan las reglas automáticamente.

Este enfoque funciona especialmente bien con problemas que son fáciles de resolver para los seres humanos, pero más difíciles de programar de forma explícita. Imagina que te encargaran detectar qué correos electrónicos deben clasificarse como spam. Con solo abrir y leer el correo electrónico, tu conocimiento y experiencia tras haber visto miles de correos electrónicos no deseados te permitirían identificar rápidamente cuáles son spam.

Pero, ¿cómo escribirías el código para identificar estos patrones? Tendrías que establecer reglas para frases sospechosas, patrones de remitentes, peculiaridades de formato o miles de otros factores. Incluso después de establecer todas estas reglas, los spammers evolucionarían constantemente sus tácticas, lo que les permitiría eludir tus reglas. El

ML evita esta complejidad aprendiendo de los ejemplos de spam y correos electrónicos legítimos, adaptándose automáticamente a medida que se identifican nuevos patrones.

Para los desarrolladores, el ML suele presentarse en tres tipos principales de problemas:

Clasificación

Determinar a qué categoría pertenece algo. Piensa en ello como responder a la pregunta «¿Qué es esto?». Algunos ejemplos son la detección de spam (spam frente a legítimo), el diagnóstico médico (enfermedad frente a salud) o la moderación de contenidos (apropiado frente a inapropiado). El algoritmo aprende a reconocer patrones que distinguen entre categorías.

Regresión

Predecir valores numéricos. En lugar de categorías, te preguntas «¿Cuánto?» o «¿Cuántos?». Algunos ejemplos son la predicción de los precios de las viviendas en función de su ubicación y características, la estimación de los plazos de entrega o la previsión de las cifras de ventas. El algoritmo aprende las relaciones entre las características de entrada y los resultados numéricos.

Agrupación

Agrupar elementos similares sin conocer los grupos de antemano. Esto responde a la pregunta «¿Qué agrupaciones naturales existen?». Algunos ejemplos son la segmentación de clientes (encontrar diferentes perfiles de compradores), organizar grandes colecciones de documentos o identificar

patrones de comportamiento de los usuarios. A diferencia de la clasificación, no le dices al algoritmo qué grupos debe buscar: él los descubre.

Como desarrollador, lo más habitual es que utilices el aprendizaje automático a través de API y modelos preentrenados, en lugar de crear algoritmos desde cero. A continuación te mostramos cómo puedes integrar el aprendizaje automático en una aplicación típica:

```
// Using a pre-trained ML model through an API
public class EmailClassifier {
    public boolean isSpam(String emailContent) {
        MLPrediction prediction = mlService.classify(
            emailContent,
            "spam-detection-model"
        );

        return prediction.getConfidence() > 0.85
            && prediction.getLabel().equals("spam");
    }
}
```

Deep Learning

El Deep Learning es un subconjunto del ML que utiliza redes neuronales con múltiples capas, de ahí el término «profundo». Pero, ¿qué es una red neuronal? Piensa en ella como un modelo simplificado inspirado en la forma en que el cerebro humano procesa la información. Tu cerebro tiene neuronas interconectadas (aproximadamente 86 000 millones) que se transmiten señales entre sí. Una red neuronal contiene

nodos que imitan esta funcionalidad (neuronas artificiales) y transmite datos a través de esas conexiones.

En una red neuronal, las capas actúan como una capa de procesamiento, donde a cada una se le asigna una tarea específica. En una red neuronal profunda, puedes tener una capa de entrada que recibe los datos sin procesar y varias capas ocultas que procesan y transforman esos datos. Cuando las capas terminan de procesar los datos, una capa de salida produce un resultado final. Estas redes contienen múltiples capas (normalmente, tres o más las hacen «profundas»), lo que les permite aprender patrones complejos.

Estas redes son muy buenas para encontrar patrones complejos en los datos. Si el aprendizaje automático tradicional enseña a un ordenador a reconocer patrones, Deep Learning le da la capacidad de captar matices y contexto. En el reconocimiento facial, la primera capa puede detectar perímetros y líneas simples, la siguiente capa puede combinarlos para reconocer formas básicas y las capas más profundas pueden identificar características más complejas como los ojos o las estructuras faciales. La capa final lo reúne todo para clasificar la imagen completa como «Dan» o «Nate».

Como desarrollador, lo más habitual es que interactúes con el Deep Learning a través de modelos preentrenados, en lugar de crear redes neuronales desde cero. Se trata de modelos que ya han sido entrenados con conjuntos de datos masivos y que pueden integrarse en tus aplicaciones. Veamos un ejemplo práctico de cómo se puede utilizar un modelo de reconocimiento de imágenes preentrenado en una aplicación Java:

```
// Using a pretrained deep learning model
public class ImageClassifier {
    public String identifyImage(byte[] imageData) {
        DeepLearningModel model = ModelLoader.load("face-recognition-v1");
        return model.classify(imageData);
    }
}
```

IA generativa

La IA generativa (*GenAI*) representa un cambio fundamental en lo que la IA puede hacer. Mientras que la IA tradicional analiza y categoriza los datos existentes, la GenAI crea nuevos contenidos que antes no existían. Si el aprendizaje automático es como enseñar a un ordenador a reconocer cómo es una buena pintura, la GenAI le enseña a crear nuevas pinturas originales. Esto es posible porque los modelos de IA se entrenan con enormes cantidades de datos de múltiples modalidades, todos ellos creados por humanos. Los modelos aprenden los patrones, estilos y estructuras de cómo crean los humanos y, a continuación, utilizan esos patrones para generar nuevos contenidos que parecen humanos.

La GenAI funciona con múltiples tipos de contenido, cada uno de los cuales abre nuevas posibilidades para los desarrolladores:

Texto y código

: La IA puede escribir artículos, documentación, funciones de código e incluso aplicaciones completas basándose en descripciones. Esto incluye desde la generación de mensajes de

error fáciles de entender hasta la creación de datos de prueba realistas.

Imágenes

Crea ilustraciones personalizadas, genera iconos que se adapten al estilo de tu aplicación, produce imágenes de marketing o crea imágenes de marcador de posición relacionadas con tu contenido. La IA entiende conceptos como «un icono de robot amistoso con un estilo de diseño plano y moderno».

Audio

Genera voz natural para funciones de accesibilidad, crea música de fondo para aplicaciones o produce efectos de sonido.

Vídeo

Produce vídeos de demostración, crea explicaciones animadas de conceptos complejos o genera contenido de marketing. La IA puede comprender solicitudes como «crea un vídeo de 30 segundos que muestre cómo utilizar esta función de la aplicación móvil».

A continuación, se muestra un ejemplo de cómo puedes utilizar GenAI en tu flujo de trabajo de desarrollo:

```
// Using generative AI to create test data
public class TestDataGenerator {

    private final ChatClient chatClient;

    public TestDataGenerator(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }
```

```
    public List<User> generateTestUsers(int count) {
        var userList = chatClient.prompt()
                .user( (u) -> {
                    var prompt = """
                        Generate {n} realistic user profiles with
                        name, email, age, and interests""";
                    u.text(prompt).param("n", count);
                })
                .call()
                .entity(UserList.class);

        return userList.users();
    }
}
```

NOTA

Aunque GenAI crea contenido nuevo, recuerda validar sus resultados. El código generado debe someterse a pruebas, los datos generados deben revisarse y cualquier contenido dirigido a los usuarios debe comprobarse para garantizar su precisión y adecuación.

Aunque la mayoría de los desarrolladores de software no participan directamente en el desarrollo de algoritmos de aprendizaje automático ni en la creación de modelos lingüísticos fundamentales, interactúan regularmente con tecnologías de IA a través de API y servicios. Comprender estos conceptos básicos te ayuda a crear mejores aplicaciones y a trabajar bien con los expertos en IA de tus equipos.

Modelos de lenguaje grandes

Los modelos de lenguaje grandes (*LLMs*) combinan los conceptos que hemos discutido en este capítulo hasta ahora. Utilizan redes neuronales de Deep Learning y son un tipo líder de GenAI. Estos modelos derivan patrones a partir de grandes cantidades de datos para comprender y generar contenido similar al humano. ChatGPT, Claude y GitHub Copilot funcionan con LLMs.

Lo que hace que los LLMs sean especialmente potentes es su capacidad para comprender el contexto y los matices del lenguaje humano. A diferencia de la IA anterior, que podía reconocer palabras clave, los LLMs pueden comprender el significado, mantener conversaciones e incluso entender información implícita. Por eso ChatGPT puede ayudarte a depurar código, Claude puede explicar conceptos complejos y GitHub Copilot puede sugerir completaciones de código relevantes.

Los LLMs destacan en tareas relacionadas con la comprensión y la generación del lenguaje. A continuación, se muestra un ejemplo práctico de cómo se puede utilizar un LLM para generar documentación automáticamente. Este ejemplo toma el código como entrada y genera comentarios JavaDoc, que documentan el propósito y el uso del código para los desarrolladores:

```
// Integrating an LLM into your application to generate documentation
public class CodeDocumentationGenerator {
    private final LLMService llm;

    public String generateDocumentation(String code) {
```

```
            String prompt = String.format(
                "Generate JavaDoc comments for this method:\n%s",
                code
            );

            return llm.complete(prompt, new CompletionOptions()
                .setMaxTokens(200)
                .setTemperature(0.3)  // Lower = more focused output
            );
        }
    }
```

NOTA

La *temperatura* en la configuración de LLM controla la aleatoriedad. Los valores más bajos (0,0-0,5) producen resultados más predecibles, mientras que los valores más altos (0,7-1,0) aumentan la creatividad. Para la generación de código, mantén temperaturas más bajas. Las temperaturas más altas aumentan la creatividad, pero también elevan el riesgo de alucinaciones, que son respuestas que suenan seguras pero que son incorrectas. Para tareas basadas en hechos, las temperaturas más bajas ayudan a mantener la precisión.

Comprender las capacidades y limitaciones de la IA

Desde sus inicios, la IA ha evolucionado significativamente, pero a pesar de sus capacidades actuales, tiene importantes limitaciones que debes comprender. Ser consciente de estas limitaciones desde el princi-

pio te ayudará a reconocer mejor cuándo y cómo utilizar la IA de manera eficaz en tus tareas diarias.

En esta sección se exploran los puntos fuertes de la IA, lo que te ayudará a identificar cuándo incorporarla a tus flujos de trabajo. Igualmente importante es que examinarás las limitaciones de los LLMs, lo que te proporcionará información sobre las áreas en las que quizá debas actuar con un poco más de cautela.

En qué destaca la IA para los desarrolladores de software

La IA está mejorando la eficiencia y la productividad en muchas profesiones, pero ¿cómo te afecta a ti como desarrollador de software? Veamos algunos ejemplos de cómo la IA puede automatizar las tareas rutinarias que te ralentizan cada día. Liberarte de estas tareas te permite centrarte en trabajos más significativos y mejorar tu felicidad general:

Tareas de codificación repetitivas

> Las tareas de codificación repetitivas son aquellas en las que la IA puede ofrecerte el mayor retorno de la inversión (ROI). La IA puede servir como un socio de codificación inteligente, ayudándote a implementar patrones similares en múltiples archivos, crear API consistentes e incluso asistir en el desarrollo de funciones completas. Esto puede ayudarte a liberar tiempo, permitiéndote centrarte en los problemas únicos de tu aplicación en lugar de escribir código repetitivo.

Scripts de automatización y tareas de «enlace»

Esta es un área en la que la IA puede aportar valor a tu flujo de trabajo diario. Como desarrollador, a menudo tienes que ocuparte de tareas operativas rutinarias, como descargar archivos de servidores, organizar directorios o automatizar pasos de implementación. La IA es muy buena generando scripts de shell, archivos bash y scripts de automatización para estas actividades «humanas».

Explicación del código

La explicación del código es una de las características más valiosas de la IA para los desarrolladores que aprenden un nuevo lenguaje, marco de trabajo o se incorporan a una nueva base de código. Con acceso a una base de código, puedes hacer preguntas a la IA como: «¿Qué hace esta aplicación a alto nivel?», «¿Cuáles son los puntos de entrada clave?», «¿Cuáles son las principales dependencias?». Cuando aprendas algo nuevo, también puedes pedirle a la IA que cree un plan de aprendizaje completo para el tema.

Generación de documentación

Esta es otra área en la que la IA destaca. Seamos realistas, a los desarrolladores no les suele gustar escribir documentación a menos que sea necesario. La IA puede ayudar a generar documentación de código en todos los niveles, desde métodos y clases individuales hasta documentación completa de proyectos. También puede convertir la documentación de un formato (como Markdown) a otro (como HTML).

Sugerencias de refactorización y optimización

Las sugerencias de refactorización y optimización son como tener a un ingeniero senior que revisa tu código y te da su opinión. Puede identificar cuellos de botella, sugerir implementaciones más limpias y recomendar características o bibliotecas de lenguaje modernas que podrían mejorar tu código.

Generación de casos de prueba

Esta es otra área en la que la IA puede intervenir y permitirte centrarte en problemas más creativos y complejos. Al igual que la documentación, las pruebas son otra área que suele descuidarse y que puede beneficiarse de la ayuda de la IA. Incluso con un conjunto de pruebas completo, es posible que se pasen por alto pruebas para casos de perímetro importantes. Cuando se le proporciona contexto sobre tu aplicación, la IA puede generar pruebas que se ajusten al estilo y los patrones de tu conjunto de pruebas existente.

Conversión de código

La conversión de código ayuda a los desarrolladores a traducir código entre lenguajes, marcos y tipos de datos. Esto funciona especialmente bien con lenguajes de alto nivel como Python, JavaScript o Java, pero tiene sus limitaciones. Puede ser menos fiable para lenguajes de bajo nivel, código específico de hardware o sistemas con dependencias complejas. Recuerda siempre probar a fondo el código convertido, ya que la IA puede pasar por alto detalles sutiles pero críticos.

Diseño de la interfaz de usuario y creación de maquetas

El diseño de la interfaz de usuario y la creación de maquetas permiten a la IA generar rápidamente diseños de interfaz de usuario, estructuras de componentes e incluso prototipos interactivos basados en tus descripciones. La IA puede crear diseños HTML/CSS, sugerir patrones de diseño responsivos y ayudarte a visualizar diferentes enfoques de interfaz antes de comprometerte con la implementación.

Comprensión de las características del lenguaje y del marco de trabajo

Con el prompt adecuado, los LLMs pueden ahorrarte una enorme cantidad de tiempo buscando documentación. Pedir a un chatbot que te explique una API o los matices de una biblioteca puede ser como tener a tu alcance a una pareja infinitamente paciente y experimentada.

Las limitaciones actuales de la IA

Aunque la IA puede ayudarnos a ser más productivos en muchas áreas, no es perfecta. Es importante comprender cuáles son las deficiencias de la IA para utilizarla de forma eficaz sin comprometer la calidad de tu código.

Al revisar las siguientes limitaciones, es fundamental distinguir entre el modelo y el producto. Cuando utilizas un chatbot, estás interactuando con un producto que se encuentra encima de un modelo específico y se comunica con él. Estas limitaciones existen principalmente a nivel

de modelo, aunque los productos suelen aumentar sus capacidades para abordarlas. Por ejemplo, si le preguntas a ChatGPT por el precio actual de tus acciones favoritas, no sabrá la respuesta porque se ha entrenado con un conjunto de datos que solo llega hasta una fecha determinada. Sin embargo, ChatGPT puede invocar una herramienta para buscar en la web la información que necesita.

La falta de conocimiento en tiempo real es una limitación importante de los sistemas actuales de IA. No pueden acceder a documentación actualizada, comprender herramientas o marcos recientes ni procesar eventos en tiempo real. Estas limitaciones restringen su capacidad para proporcionar las respuestas más precisas y relevantes.

Las alucinaciones representan una de las limitaciones más peligrosas de la IA. Los sistemas de IA no proporcionan respuestas incorrectas de forma intencionada, pero pueden generar respuestas basadas únicamente en sus datos preentrenados y en el contexto que se les proporciona. Pueden proporcionar con confianza respuestas inexactas que parecen plausibles, pero que son incorrectas. Esto ocurre porque la IA genera respuestas basadas en patrones de los datos de entrenamiento, no en el conocimiento de los hechos.

La falta de información contextual sobre tu código base significa que la IA no puede comprender la arquitectura de tu proyecto, las convenciones existentes o las reglas de negocio. Sin este contexto, genera código de forma aislada, con dificultades para integrarse adecuadamente en tu sistema existente o cumplir con los estándares y preferencias de codificación de tu equipo.

Los sesgos incorporados en los datos de entrenamiento pueden llevar a la IA a generar patrones problemáticos o a hacer suposiciones que no reflejan perspectivas diversas. Los modelos de IA se entrenan con repositorios de código y documentación existentes, que pueden contener prácticas obsoletas, sesgos culturales o enfoques que no tienen en cuenta la accesibilidad y la inclusividad. Esto puede dar lugar a la generación de código que, sin quererlo, excluye a determinados grupos.

La incapacidad de comprender los requisitos empresariales limita la capacidad de toma de decisiones de la IA. No puede sopesar las ventajas y desventajas entre el rendimiento y la mantenibilidad, comprender las necesidades de cumplimiento normativo ni tomar decisiones que requieran conocimientos especializados sobre tu sector específico o tu base de usuarios.

El rendimiento inconsistente en diferentes ámbitos significa que la IA destaca con los lenguajes populares y los patrones comunes, pero tiene dificultades con las tecnologías nicho, los marcos más nuevos o los ámbitos especializados en los que los datos de entrenamiento son limitados.

Compartir código o datos con herramientas de IA conllev*ariesgos para la privacidad y la seguridad*. Muchas plataformas de IA almacenan el historial de conversaciones, utilizan las entradas para mejorar sus modelos o pueden exponer inadvertidamente información confidencial. Compartir código propietario, claves API, esquemas de bases de datos o lógica empresarial confidencial con herramientas de IA puede crear vulnerabilidades de seguridad y problemas de propiedad intelectual que requieren una cuidadosa consideración.

ADVERTENCIA

Trata siempre el código generado por IA con los mismos procesos rigurosos que el código escrito por un humano. Considéralo un primer borrador que necesita pruebas exhaustivas, revisiones del código y una planificación cuidadosa de la integración. El tiempo ahorrado durante la generación inicial del código debe reinvertirse en una validación exhaustiva.

Al comprender tanto las fortalezas como las limitaciones de la IA, puedes crear un plan para incorporar estas herramientas en tu flujo de trabajo de desarrollo y mantener al mismo tiempo la calidad del código. En la siguiente sección, explorarás herramientas específicas que pueden aumentar tu productividad diaria.

La IA como tu programador en pareja

Imagina tener a un colega experto sentado a tu lado, listo para responder preguntas, sugerir soluciones y ayudarte a resolver problemas. Disponible las 24 horas del día, los 7 días de la semana, nunca se cansa y nunca te juzga por hacer preguntas «obvias». Eso es exactamente lo que la IA puede ser para ti como desarrollador: el compañero de programación en pareja definitivo.

En lo que respecta a la programación, la IA es realmente buena en tareas que van desde la asistencia en línea hasta la planificación de proyectos a gran escala. En esta sección, exploraremos las categorías de asistentes de programación de IA. Examinaremos productos específicos de cada categoría que están disponibles en el momento de escribir

este libro. Aunque los productos individuales pueden aparecer y desaparecer debido al rápido avance de la IA, es probable que estas categorías fundamentales se mantengan estables.

Asistentes de chatbot independientes

Los asistentes de chatbot independientes como ChatGPT, Google Gemini y Anthropic Claude son sistemas de IA conversacional con los que interactúas a través de una interfaz de texto o voz, independiente de tu entorno de desarrollo. Piensa en ellos como tu mentor de programación siempre disponible.

Estas herramientas son excelentes para ayudarte a comprender conceptos, depurar problemas complejos y planificar decisiones de arquitectura. Úsalas cuando necesites generar código reflexivo con explicaciones, resolver problemas complejos o mantener conversaciones prolongadas sobre retos de programación.

A continuación se indican las buenas prácticas para las conversaciones centradas en el código:

- Proporciona contexto sobre lo que estás tratando de lograr.
- Comparte fragmentos de código relevantes cuando preguntes sobre problemas específicos.
- Prompt con tu perfil y quién eres. Pide explicaciones, no solo soluciones.
- Utiliza el asistente de IA para aprender nuevas tecnologías o marcos de trabajo.

Por ejemplo, si tienes una lista de objetos que deben ordenarse, es poco probable que un prompt como «¿Cómo ordeno esto?» te dé una respuesta útil. No le dice a la IA qué tipo de código necesitas. En cambio, es probable que un prompt como la siguiente sea más útil: «Tengo una lista de objetos Cliente y quiero ordenarlos por fecha de registro, de más reciente a más antiguo, utilizando Java 24».

Como continuarás viendo en «Fundamentos de la ingeniería de prompts», la calidad del prompt que le proporciones a una IA influye en gran medida en la calidad de los resultados que recibas.

Asistentes IDE en línea

Los asistentes IDE en línea son herramientas basadas en IA que funcionan directamente dentro de tu IDE y ofrecen sugerencias en tiempo real y autocompletado mientras escribes. Estas herramientas se integran en tu entorno de desarrollo a través de complementos o funciones integradas, lo que ayuda a acelerar la codificación al encargarse de las tareas rutinarias.

Entre las opciones más populares se incluyen GitHub Copilot, JetBrains AI Assistant y Amazon CodeWhisperer. La mayoría de los editores de código modernos son compatibles con estos asistentes de IA, lo que los hace accesibles a desarrolladores de cualquier nivel.

Debes utilizar asistentes en línea cuando realices lo siguiente:

- Escribir código repetitivo como getters/setters o constructores
- Implementar patrones estándar (singleton, builder, etc.)

- Generar pruebas unitarias basadas en código existente
- Crear comentarios y documentación de código
- Necesitar sugerencias rápidas sobre API y sintaxis

Si quieres sacar el máximo partido a un asistente de IA, debes tener en cuenta estos puntos:

- Revisar el código generado antes de aceptarlo.
- Utilizar las sugerencias como puntos de partida, no como soluciones definitivas.
- Prestar atención a los patrones que la IA aprende de tu base de código.
- No dejes que las sugerencias de la IA anulen tus decisiones arquitectónicas.
- Comienza con tareas sencillas y repetitivas y utilízalas gradualmente para patrones más complejos.

Veamos cómo un asistente en línea puede ayudar tanto con el código como con la documentación:

```
public class Calculator {
    // Start typing "public int add..." and AI suggests:
    public int addNumbers(int a, int b) {
        return a + b;
    }

    // Or start typing "/** Calculate..." for documentation:
    /**
     * Calculates the sum of two integers
     * @param a the first number
```

```
        * @param b the second number
        * @return the sum of a and b
        */
    }
```

CONSEJO

Con cualquier código generado por IA, debes evitar caer en la trampa de aceptarlo y seguir adelante. Pregúntate: «¿Entiendo lo que hace este código y por qué funciona así?». Si no es así, investiga por tu cuenta para comprenderlo mejor. Debes ser capaz de explicar cada línea durante una revisión del código. Recuerda: la IA es una herramienta para maximizar la productividad, no un sustituto de la comprensión. El objetivo es aprender más rápido, no evitar aprender por completo.

Entornos IDE de IA agencial

La categoría más reciente de herramientas de programación en pareja con IA incluye entornos de desarrollo basados en IA e , como Cursor, Junie y Cline. Estas herramientas van más allá de las sugerencias y participan activamente en las tareas de codificación, pudiendo escribir funciones o archivos completos en función de tus requisitos.

Estos entornos pueden comprender todo el contexto de tu código base y realizar cambios más sofisticados en varios archivos. Son especialmente potentes para refactorizar, implementar funciones y mantener la coherencia en todo tu proyecto.

A continuación se muestran algunos ejemplos de cuándo utilizar entornos agenticos:

- Refactorización a gran escala en varios archivos
- Implementación de funciones que requieren cambios en varios lugares
- Tareas de migración o modernización de código
- Cuando necesites que la IA comprenda el contexto más amplio del proyecto

Estas son las buenas prácticas cuando se trabaja con entornos agenticos:

- Comienza con requisitos claros y específicos.
- Define los requisitos no funcionales desde el principio (estilo de código, normas de seguridad, enfoque de pruebas).
- Revisa cuidadosamente todos los cambios antes de confirmarlos.
- Utiliza el control de versiones para realizar un seguimiento de los cambios generados por la IA.
- Divide las tareas grandes en partes más pequeñas y manejables.

Considera un ejemplo práctico. Imagina que tienes una aplicación de panel de control con cinco widgets existentes: WeatherWidget, StockPriceWidget, NewsWidget, CalendarWidget y TaskListWidget. Le pides a una IA agencial que cree un nuevo StatusWidget que muestre las métricas de estado del sistema.

Así es como un agente de IA aborda esta tarea:

Fase de análisis

La IA examina primero tus widgets existentes para comprender los patrones:

- Revisa la estructura básica de la clase Widget
- Identifica métodos comunes como `render()`, `updateData()` y `getConfig()`
- Observa el uso coherente de un patrón de servicio de datos
- Observa las convenciones de estilo y la organización de los componentes

Fase de planificación

La IA crea un plan:

- «Necesito crear StatusWidget ampliando la clase Widget base».
- «Necesitaré un StatusService para obtener las métricas del sistema».
- «El widget debe seguir el mismo patrón de actualización que los demás».
- «Tendré que añadir el nuevo widget al registro del panel de control».

Implementación

Solo después de este análisis, la IA comienza a escribir código, creando lo siguiente:

- *StatusWidget.java* siguiendo los patrones establecidos
- *StatusService.java* coincide con la arquitectura de servicios existente
- Configuración del panel de control actualizada para incluir el nuevo widget
- Estilo coherente que coincide con los demás widgets

Esto demuestra el poder de la IA agencial: no solo genera código, sino que comprende el contexto de tu proyecto y mantiene la coherencia en todo tu código base.

CONSEJO

Recuerda: tú eres el piloto, no el pasajero. Las herramientas de IA son asistentes poderosos, pero tú sigues siendo responsable de comprender el código, tomar decisiones arquitectónicas y garantizar la calidad. Usa la IA para ampliar tus capacidades, no para sustituir tu pensamiento. En última instancia, tú eres responsable de todo el código de tu proyecto, independientemente de si lo has generado tú o la IA.

La clave del éxito con cualquier herramienta de codificación de IA es comprender sus puntos fuertes y sus limitaciones, y luego elegir la herramienta adecuada para cada situación. A medida que crezcas como desarrollador, estos asistentes de IA evolucionarán contigo, convirtiéndose en socios más sofisticados en tu viaje por la codificación.

Fundamentos de la ingeniería de prompts

Cuando se trata de convertirse en un desarrollador de software que aprovecha la IA, la ingeniería de prompts es una de las habilidades más valiosas que debes comprender. Tanto si eres un principiante como un usuario experimentado, dominar la creación de prompts claros y eficaces mejorará drásticamente tus resultados al trabajar con asistentes de IA, herramientas de generación de código e IDE agenticos.

La realidad es que muchos desarrolladores tienen dificultades con las herramientas de IA, no porque la tecnología sea deficiente, sino porque no han aprendido a comunicar sus necesidades de manera eficaz. Esta habilidad suele manifestarse en el mundo real al redactar requisitos claros para un compañero de equipo o al documentar tu código. Cuando trabajes con IA, debes tratarla como si fuera una conversación con un compañero de trabajo y utilizar una comunicación clara y reflexiva.

¿Qué es la ingeniería de prompts?

Como has aprendido anteriormente en este capítulo, hay algunos términos que hacen que trabajar con IA resulte un poco intimidante, y *la ingeniería de prompts* es otro ejemplo de ello. La ingeniería de prompts consiste simplemente en aprender a comunicarse de forma eficaz con la IA. Piensa en ello como aprender a utilizar eficazmente un motor de

búsqueda. Es una habilidad que todo el mundo necesita desarrollar, no algo reservado a los especialistas.

Imagina que estás en la oficina y te diriges a un desarrollador junior en el cubículo de al lado para pedirle que «arregle el error de inicio de sesión». Sin contexto, esta sencilla tarea puede convertirse en un problema complejo. Es posible que pierda tiempo buscando en el sistema de tickets los problemas abiertos. Dado que hay varios formularios de inicio de sesión, ¿cuál hay que arreglar? Podría pasar horas probando el sistema de inicio de sesión sin encontrar nada malo. Pero, ¿y si en lugar de eso dijeras: «El formulario de inicio de sesión del panel de control del usuario no valida correctamente los formatos de correo electrónico, lo que permite a los usuarios enviar direcciones de correo electrónico no válidas que están causando problemas posteriores»? Obtendrías resultados mucho mejores con esta interacción.

El mismo principio se aplica a la IA. La calidad de tu prompt está directamente relacionada con la calidad de la respuesta de la IA.

Técnicas esenciales de ingeniería de prompts

Para sacar el máximo partido a la IA no basta con hacer preguntas, sino que hay que hacer las preguntas adecuadas de la forma correcta. En esta sección, descubrirás técnicas que te ayudarán a transformar tu forma de interactuar con las herramientas de IA. En lugar de obtener respuestas genéricas que requieren numerosas iteraciones, aprenderás a escribir prompts que produzcan de forma sistemática los resultados que buscas.

La comunicación clara es clave

Si quieres obtener buenos resultados de la IA, debes ser específico. Esta es una técnica que aprenderás con el tiempo, repitiendo tus prompts cuando no obtengas el resultado deseado. Por ejemplo, considera los siguientes prompts.

Instrucción incorrecta:

```
Write some Java code for sorting
```

Este prompt es demasiado vaga. No especifica qué clasificar, cómo clasificarlo ni qué estructura de datos utilizar. La IA tiene que adivinar si quieres clasificar números, cadenas, objetos o algo completamente diferente.

Buena prompt:

```
Write a Java method that sorts an ArrayList of Employee objects by their salary
in descending order. Include error handling for null inputs.
```

Este prompt le da a la IA todo lo que necesita: la estructura de datos específica (`ArrayList`), lo que estás ordenando (`Employee` objects), los criterios de ordenación (salary), el orden (descending) e incluso un caso extremo a tener en cuenta (null inputs).

La estructura determina el éxito

Un prompt bien estructurado puede ayudar a la IA a comprender no solo lo que quieres, sino también cómo quieres que se entregue el resultado. Por ejemplo, compara los dos prompts siguientes.

Solicitud incorrecta:

```
Explain database indexing
```

Este prompt no ofrece ninguna orientación sobre la profundidad, la perspectiva o el formato. La IA podría responder con cualquier cosa, desde una sola frase hasta una explicación de informática de nivel universitario.

Buena prompt:

```
As an experienced Java developer, explain database indexing in 3 parts:

1. What indexes are and why they matter for performance
2. When to use clustered vs non-clustered indexes
3. One practical example of creating an index in PostgreSQL

Keep each section to 2-3 sentences and include one code example.
```

Este prompt proporciona una estructura clara (tres partes numeradas), especifica la perspectiva del público (desarrollador de Java), establece

las expectativas de longitud (de dos a tres frases cada una) y solicita un ejemplo concreto.

Piensa en ello como enseñar, no como dar órdenes

Si eres un buen mentor para un desarrollador junior, no te acercarías a él para exigirle un resultado en una tarea específica. En cambio, le darías todo el contexto posible y le enseñarías mediante ejemplos que ya podrían existir en tu código base. En lugar de exigir resultados, guía a la IA a través de tu proceso de pensamiento. Este enfoque, denominado *«cadena de pensamiento»*, da *lugar* a respuestas más precisas. Por ejemplo, compara los dos siguientes prompts.

Prompt incorrecto:

```
Optimize this code.

[paste your code here]
```

Este prompt no proporciona ningún contexto sobre lo que significa «optimizar» para ti. La IA no sabe si quieres una ejecución más rápida, un menor uso de memoria, una mejor legibilidad o algo completamente diferente.

Buena prompt:

```
I need to optimize this Java method that searches through a large dataset.
First, help me identify the current time complexity. Then suggest specific
```

```
improvements. Finally, show me the refactored code with comments explaining
the performance gains.

[paste your code here]
```

Este prompt divide la tarea en pasos lógicos, especifica el objetivo de optimización (rendimiento para grandes conjuntos de datos) y solicita explicaciones junto con los cambios en el código.

Consejos prácticos para una mejora inmediata

A continuación se presentan técnicas prácticas de ingeniería de prompts que puedes utilizar de inmediato para mejorar los resultados generados por la IA:

Sé específico sobre el contexto

: Proporciona siempre información contextual relevante. ¿Se trata de un proyecto nuevo o de uno ya existente que no puede aprovechar las buenas prácticas modernas? ¿Es un proyecto personal o una aplicación crítica para el trabajo?

Utiliza ejemplos

: Muéstrale a la IA cómo es un buen resultado. Si quieres que el código tenga un formato determinado, proporciona un ejemplo.

Proporciona contexto sobre tu entorno

: Menciona tu versión de Java, los marcos que utilizas o las restricciones con las que trabajas.

Especifica una función

Dale a la IA una personalidad o experiencia específica que encarnar. Por ejemplo, «Actúa como un arquitecto Java sénior que revisa este código» o «Como experto en optimización del rendimiento, analiza esta consulta». Esto ayuda a enmarcar la respuesta con el nivel adecuado de detalle y perspectiva.

Repite y perfecciona

No esperes la perfección en el primer intento. Utiliza la respuesta de la IA para perfeccionar tu prompt y acercarte al resultado deseado.

Utiliza la IA para mejorar tus prompts

Cuando no obtengas los resultados que deseas, pide a la IA que te ayude a redactar un mejor prompt. Prueba algo como: «Estoy intentando que me ayudes a depurar este problema de rendimiento, pero tu respuesta no era exactamente lo que necesitaba. ¿Puedes sugerirme cómo debería reformular mi solicitud para obtener pasos de depuración más específicos?»

Guarda las buenas indicaciones

Cuando redactes un prompt que funcione bien, guárdalo. Es probable que necesites solicitudes similares en el futuro. Para obtener consejos y trucos más detallados sobre este tema, consulta «Gestión del conocimiento personal».

Técnicas avanzadas de ingeniería de prompts

Una vez que dominas los conceptos básicos, las técnicas avanzadas pueden ayudarte a abordar tareas de desarrollo más complejas. Exploraremos dos categorías principales de técnicas avanzadas: técnicas de estructuración que te ayudan a dar formato y enmarcar tus solicitu-

des, y técnicas de organización que te ayudan a gestionar prompts complejos y multiparte.

Técnicas de estructuración

Veamos algunas técnicas comunes de prompting y cómo utilizarlas.

El prompt sin ejemplos (zero-shot prompting) consiste en pedir a la IA que realice una tarea sin proporcionar ningún ejemplo. No hay nada de malo en este enfoque y, en ocasiones, te dará exactamente lo que buscas. Por ejemplo:

```
Create a Java class that implements the Observer pattern for a
stock price monitoring system.
```

El prompting de un solo intento (one-shot prompting) proporciona un único ejemplo para establecer un patrón que deseas que siga la IA. Por ejemplo, supongamos que le proporcionas a la IA lo siguiente:

```
// Here's an example of the coding style I prefer:

public Optional<User> findUserByEmail(String email) {
    if (email == null || email.trim().isEmpty()) {
        return Optional.empty();
    }
    return userRepository.findByEmail(email.toLowerCase());
}

// Now create a similar method called findUserById that takes a Long id parameter.
```

En el ejemplo, le has enseñado a la IA cuáles son tus preferencias a la hora de escribir un tipo concreto de método. Basándose en eso, deberías poder obtener métodos similares generados para ti que sigan ese estilo.

Las indicaciones de pocos ejemplos enseñan a los modelos de IA a través de ejemplos en lugar de solo instrucciones. En lugar de explicar lo que deseas, le muestras a la IA varios ejemplos del patrón de entrada-salida deseado. Por ejemplo, supongamos que deseas que la IA clasifique el sentimiento de las opiniones de los clientes:

```
Classify the sentiment of these customer reviews:

Example 1: "The software is intuitive and saves me hours of work" → Positive
Example 2: "Great documentation and excellent support team" → Positive
Example 3: "Buggy interface and crashes frequently" → Negative

Now classify: "The new features are helpful but the UI is confusing"
```

Si recuerdas nuestra terminología de IA, esto se reduce a *la clasificación* y a proporcionar al modelo suficientes ejemplos para que pueda clasificar correctamente los ejemplos futuros.

Técnicas de organización

Las técnicas de organización te ayudan a estructurar las solicitudes para obtener respuestas mejores y más precisas de los sistemas de IA. Al organizar claramente tus prompts, reduces la incertidumbre y ayudas a la IA a centrarse en lo que más importa.

Las etiquetas XML (y otros formatos de estructuración como Markdown o JSON) ayudan a estructurar solicitudes complejas separando claramente los diferentes tipos de información. Esto facilita que la IA comprenda para qué sirve cada dato y cómo priorizarlo. Por ejemplo:

```
<task>
Create a RESTful API endpoint for user management
</task>

<requirements>
- POST /users for creating users
- Include validation for email and password
- Return appropriate HTTP status codes
- Use Spring Boot annotations
</requirements>

<constraints>
- Java 17
- Spring Boot 3.0
- No external dependencies beyond Spring starter
</constraints>
```

La estructura XML ayuda a la IA a distinguir entre lo que quieres construir (tarea), lo que debe incluir (requisitos) y las limitaciones con las que debe trabajar (restricciones). Sin esta estructura, toda la información se mezcla y la IA podría pasar por alto detalles importantes o priorizarlos incorrectamente.

La descomposición de tareas divide los problemas complejos en partes manejables. Esto es algo que nosotros, como ingeniería de software,

hacemos constantemente. Del mismo modo que dividirías esto en tareas más pequeñas para ti o para un desarrollador junior, deberías hacer lo mismo con la IA para mejorar tus resultados. Esto es importante porque los modelos de IA pueden verse abrumados por solicitudes grandes y producir respuestas incorrectas o incompletas. Dividir tu solicitud en fases te permite obtener un análisis más exhaustivo de cada componente y realizar ajustes sobre la marcha. Por ejemplo:

```
I'm building a file processing system in Java. Help me break this down:

Phase 1: Design the file reading strategy (stream vs batch)
Phase 2: Create the data validation logic
Phase 3: Implement error handling and logging
Phase 4: Add unit tests

Start with Phase 1 - analyze the pros and cons of each approach
for processing 1GB+ files.
```

La clave para ser productivo con las herramientas de IA no consiste en memorizar marcos o técnicas, sino en desarrollar hábitos de comunicación claros y comprender qué enfoques funcionan mejor. Empieza por escribir prompts específicos que enseñen al sistema el resultado deseado. Proporciona contexto sobre ti y tu proyecto y estate dispuesto a perfeccionar tu enfoque mediante la iteración.

Ahora que entiendes cómo sacar el máximo partido a la IA, exploremos cómo podría ser el futuro del desarrollo de software.

Cómo podría influir la IA en la ingeniería de software

¿Cómo se vislumbra el futuro del trabajo para los ingenieros de software? Aunque nadie puede predecir con certeza lo que nos depara el futuro, no hay duda de que la IA desempeñará un papel importante en el desarrollo de software. A medida que la IA sigue creciendo y cambiando con el tiempo, es importante tener en cuenta qué te diferencia de otros ingenieros. A medida que la IA se vuelve cada vez más competente en la escritura de código, tus cualidades únicas y tus valiosas habilidades sociales serán más importantes que nunca, ya sean tus habilidades de colaboración, empatía, capacidad de tutoría u otras características distintivas.

En esta sección se analiza cómo la IA está transformando el papel del desarrollador. En lugar de sustituir a los desarrolladores, la IA sirve como una potente herramienta que mejora la productividad y transforma la forma en que escribimos y revisamos el código. Aprende a aprovechar la IA para resolver problemas de forma más eficaz, al tiempo que destacas tus habilidades actuales.

¿La IA me quitará el trabajo?

En este capítulo, has visto cómo la IA puede automatizar tareas, generar código, escribir pruebas y crear documentación. A medida que estas capacidades mejoran, los desarrolladores de software de todos los niveles se preguntan: «¿La IA me quitará el trabajo?». Para responder

a esta pregunta, podemos extraer algunas conclusiones razonables a partir de los patrones históricos y las tendencias actuales.

Como has aprendido a lo largo de este libro, la ingeniería de software implica mucho más que escribir código. En el capítulo 2, aprendiste que pasarás mucho más tiempo leyendo código que escribiéndolo. Esta habilidad fundamental, la capacidad de entender y comprender el código, sigue siendo esencial, ya sea que ese código provenga de otro desarrollador o de la IA.

Este es solo un ejemplo de las muchas cosas que hacés como desarrollador de software además de escribir código. Aprovechemos esta oportunidad para repasar algunas de las tareas que componen tu función. Antes de escribir cualquier código, tenés la responsabilidad de convertir requisitos incompletos y ambiguos en sistemas que funcionen. Como aprendiste en el capítulo 9, tomás decisiones («depende») sobre las compensaciones entre características como el rendimiento y la facilidad de mantenimiento. Te mueves en la política de la oficina, asesoras a desarrolladores junior y explicas conceptos técnicos a partes interesadas sin conocimientos técnicos. Entiendes no solo *cómo* construir algo, sino *por qué* debe construirse y *a quién* sirve.

La IA se ha entrenado con millones de ejemplos, tanto buenos como malos, y puede generar código nuevo basándose en estos datos preentrenados. Pero no entiende el modelo de negocio específico de tu empresa, las necesidades únicas de tus usuarios o por qué la «simple petición» del director general es en realidad una completa pesadilla arquitectónica. No puede estar presente en una reunión y darse cuenta de que los requisitos del director de producto no coinciden con lo que

prometió el equipo de ventas. Estos elementos humanos del desarrollo de software no son casos extremos, son el trabajo.

> *La IA no sustituirá a los desarrolladores, pero los desarrolladores que utilicen la IA sustituirán a los que no lo hagan.*
>
> —Jeff Atwood (atribuido), desarrollador de software, autor, bloguero y empresario

¿Recuerdas el telar Jacquard de la introducción de este capítulo? Los tejedores que prosperaron no fueron los que lucharon contra la tecnología, sino los que aprendieron a manejarla, diseñaron patrones para ella y la utilizaron para crear cosas que antes eran imposibles.

Hoy en día se está produciendo un patrón similar para los desarrolladores de software. La IA está eliminando las tareas mundanas que los programadores suelen encontrar tediosas. Esto abarca desde escribir código repetitivo hasta crear pruebas repetitivas, crear documentación y mucho más. Puedes ver esto como una amenaza o puedes verlo como lo que es: una liberación. Cuando dedicas menos tiempo a tareas poco estimulantes, tienes más tiempo para el trabajo creativo y estratégico que hace que el desarrollo de software sea emocionante.

Si te detienes a observar la evolución de la programación, verás que la industria ha pasado de los lenguajes ensambladores a los lenguajes de alto nivel, de los simples editores de texto a las potentes herramientas de desarrollo, los sofisticados IDE y los procesos automatizados de CI/CD. A pesar de todos los avances tecnológicos que podrían haber de-

jado obsoletos a los programadores, la demanda de desarrolladores ha crecido año tras año.

Los desarrolladores que tendrán éxito en el futuro de la IA son aquellos que ven la IA como su socio de codificación, un socio que tiene superpoderes y nunca se cansa, que tiene el conocimiento histórico de Internet y que puede realizar tareas básicas para que tú puedas trabajar en problemas más grandes y complejos. Lo que te hace valioso no es tu capacidad para escribir un bucle «for» cuando se te pide o saber cómo implementar un algoritmo de ordenación por burbuja, sino saber *cuándo* utilizarlos, *por qué* son importantes y *cómo* encajan en la aplicación más amplia que estás creando.

¿Entonces, la IA eliminará algunos puestos de trabajo? Es posible. ¿Pero eliminará la mayoría de los puestos de trabajo? No a corto plazo. Algunas funciones se rediseñarán, y eso está bien. Si eres capaz de aceptar ese cambio, te sentirás más productivo y capaz de crear cosas que hace unos años habrían sido imposibles. La cuestión no es si la IA te sustituirá, sino si serás uno de los desarrolladores que aprovechan esta nueva y potente herramienta o uno de los pocos que siguen aferrándose a su capacidad para escribir código como si estuviéramos en 2010.

Entonces, si la IA no nos está reemplazando, ¿cómo cambiará exactamente nuestro trabajo diario? Exploremos cómo está evolucionando el papel del desarrollador y qué nuevas habilidades te diferenciarán del resto.

Reseñas de código de Vibe

El 5 de febrero de 2025, en , Andrej Karpathy, una de las figuras más influyentes en el campo de la IA y el Deep Learning, publicó un artículo en X en el que resumía sus experiencias actuales en el desarrollo de software y acuñó el término *«vibe coding» (codificación* vibrante). El término « » en sí mismo es deliberadamente irónico. Karpathy lo acuñó para destacar las prácticas de desarrollo problemáticas, no para respaldarlas.

¿Qué es la programación vibrante?

La programación vibrante se refiere a la práctica de utilizar IDE agenticos para generar aplicaciones completas a través de prompts bien elaborados. En lugar de realizar la tediosa tarea de escribir manualmente cada línea de código, los desarrolladores pueden describir lo que quieren construir y las herramientas de IA generan una cantidad significativa del código base de forma autónoma.

Este enfoque ha hecho que el desarrollo de software sea accesible a un grupo completamente nuevo de creadores. Alguien con una gran idea ya no necesita contratar a un equipo de desarrolladores ni pasar años aprendiendo a programar solo para crear un prototipo básico. Puede describir su visión a un asistente de IA y ver cómo su concepto cobra vida ante sus ojos.

Piensa por un momento en las implicaciones que esto tiene. El propietario de un restaurante que quiere crear un sistema de pedidos sencillo, un profesor que crea un cuestionario interactivo para sus alumnos

o un artista que desarrolla un sitio web con su portfolio ahora pueden hacer realidad sus ideas sin necesidad de tener experiencia en programación tradicional.

Las ventajas y los peligros

Como señaló Karpathy en su tuit original, la programación vibrante funciona bien para «proyectos de fin de semana desechables», pero la palabra clave aquí es «desechables». Hay una enorme diferencia entre crear un proyecto personal que solo tú vas a utilizar y desarrollar un software que impulse la columna vertebral de las empresas de la lista Fortune 500.

Considera estos dos escenarios:

Escenario 1

> Quieres crear un registro de gastos personales para clasificar tus gastos mensuales. Utilizas la codificación vibe para generar la aplicación durante un fin de semana. Si un error calcula mal tus gastos en café, el peor resultado es un presupuesto ligeramente inexacto.

Escenario 2

> Tu empresa necesita un sistema de nóminas que gestione a miles de empleados en varios estados con diferentes requisitos fiscales. Un error aquí podría significar que los empleados no cobren correctamente, que no se cumplan las obligaciones fiscales y que la empresa se enfrente a consecuencias legales.

Lo que está en juego no podría ser más diferente.

La realidad empresarial

Cuando leas titulares que afirman que «el 25 % del código de las grandes empresas ahora lo escribe la IA», es importante comprender cómo se traduce esto en la práctica. Este código asistido por IA no es el resultado de sesiones de programación casuales en las que se generan aplicaciones completas de forma mágica a partir de simples prompts.

Un ejemplo realista es el flujo de trabajo que han demostrado empresas como Microsoft. Un desarrollador crea un problema detallado en el que describe la situación. Un agente de codificación de IA lee ese problema, genera los cambios reales en el código y, a continuación, crea una solicitud de incorporación de cambios con esos cambios. Un desarrollador humano (o un equipo de desarrolladores) revisa el código y decide si fusionar este cambio en la base de código. La IA se encarga del trabajo rutinario de implementación, pero los humanos siguen controlando el control de calidad y la toma de decisiones.

Los entornos empresariales en los que trabajas tienen algo de lo que carecen estos guerreros de fin de semana: procesos rigurosos. No quiere decir que no se produzca cierto nivel de programación intuitiva dentro de la empresa, pero aquí existen controles y contrapesos. Cuentan con revisiones de código, marcos de pruebas, procesos de implementación y procedimientos de control de calidad. Este nuevo código generado por IA sigue teniendo que pasar por el mismo escrutinio que el código escrito por humanos. De hecho, estos procesos son más críticos que nunca.

Por qué las revisiones de código son más importantes que nunca

Aunque las herramientas de IA pueden generar código más rápido que nunca, eso no significa que tus plazos se reduzcan exponencialmente ni que no conlleven costes ocultos. El código generado (al igual que el código escrito por humanos) puede contener errores sutiles, vulnerabilidades de seguridad o decisiones de arquitectura que podrían no ser adecuadas para tu aplicación. Desarrolladores expertos como Neal Ford advierten de un posible «tsunami de código defectuoso» a medida que los equipos se apresuran a implementar soluciones generadas por IA sin los procesos de revisión adecuados.

Por eso las revisiones de código se han vuelto más críticas que nunca, y no menos importantes. Como desarrollador de esta función, debes comprender qué hace cada línea de código generado y por qué se tomaron ciertas decisiones. Como revisor, debes comprender que, independientemente de quién lo haya escrito (un humano o una máquina), necesitas prestar atención a cada detalle. ¿Prefieres descubrir el problema durante un fin de semana de guardia o dedicar tiempo a la revisión del código para detectar estos problemas?

Dado que las revisiones de código van a convertirse en una parte más importante del ciclo de vida del desarrollo de software (SDLC), hay una conclusión importante que extraer. Si vas a utilizar la IA para generar un método, una clase o una funcionalidad completa, debes comprender y explicar «tu» código. Antes de enviar una solicitud de incorporación de cambios (PR) para la revisión del código, debes revisar minuciosamente tu código y ser capaz de explicar por qué existe ese código y qué hace. No puedes usar la excusa de que la IA generó ese código y

que no fue culpa tuya. Tanto si lo escribes tú como si lo hace la IA, tú eres responsable de él y de los resultados.

La nueva mentalidad del desarrollador

Como desarrollador en esta era asistida por la IA, la clave es evitar la trampa del «vibe coding». Si bien las herramientas de IA pueden acelerar el desarrollo, el enfoque casual de «prompt-and-pray» (es decir, introducir prompt y rezar) que Karpathy satirizó representa todo lo que debes evitar. En cambio, debes pensar en ti mismo como creador y curador de código. Debes utilizar la IA de forma reflexiva, manteniendo al mismo tiempo unos estándares rigurosos.

Esto significa desarrollar las siguientes habilidades:

- Escribir prompts eficaces que generen un mejor código de IA
- Revisar y comprender rápidamente el código generado
- Identificar posibles problemas tanto en el código escrito por humanos como en el escrito por IA
- Identificar posibles sesgos o exclusividades
- Comunicarte de forma eficaz durante las revisiones de código

La programación intuitiva representa un cambio emocionante en el desarrollo de software, pero no elimina la necesidad de prácticas de desarrollo cuidadosas y reflexivas. En todo caso, estos procesos se vuelven más importantes que nunca. A medida que avanzas en tu carrera, adopta las herramientas que te hagan más eficiente y productivo, pero no pierdas de vista la responsabilidad fundamental de entregar software confiable y fácil de mantener.

La IA como multiplicador de fuerzas: de escribir código a resolver problemas

Un *multiplicador de fuerzas* es un concepto tanto militar como empresarial: es una herramienta o capacidad que amplifica tus habilidades existentes. Por ejemplo, la mayoría de la gente utiliza sus teléfonos inteligentes para hacer fotos. Las cámaras de los teléfonos inteligentes actuales son excelentes y cuentan con funciones que te ayudan a hacer fotos de alta calidad. Sin embargo, existe toda una gama de mejoras disponibles en forma de cámaras DSLR. Estas cámaras profesionales pueden costar miles de dólares y producir fotos impresionantes si sabes cómo utilizarlas.

Si te dieran una cámara de 5000 dólares y le entregaran a un fotógrafo profesional con 20 años de experiencia solo un teléfono inteligente, ¿quién tomaría mejores fotos? Probablemente ganaría el fotógrafo profesional, porque entiende conceptos como la apertura, la exposición, la iluminación y la composición, además de tener la visión artística para capturar exactamente la imagen que desea. La cámara cara es un multiplicador de fuerzas. Amplifica las habilidades existentes del fotógrafo, pero no puede crear habilidades que aún no existen.

Esto es similar al estado actual de la IA en el desarrollo de software. La herramienta no te convierte en un mejor desarrollador, del mismo modo que una cámara cara no te convierte en fotógrafo. Tus conocimientos, tu experiencia y tu capacidad para resolver problemas son lo que te diferencia. La IA es tu smartphone, una potente herramienta que amplifica lo que ya sabes.

Al fin y al cabo, independientemente de la tecnología que utilices, sigues creando software. El cambio fundamental que deben tener en cuenta los desarrolladores no es lo que crean, sino cómo emplean su tiempo en crearlo. El desarrollo de software tradicional sigue unos patrones. Es posible que dediques el 70 % de tu tiempo a escribir código repetitivo, depurar errores de sintaxis e implementar patrones repetitivos. Esto te deja un 30 % de tu tiempo para la resolución creativa de problemas y todas las demás tareas administrativas que conlleva tu trabajo.

La IA nos permite invertir esta ecuación. Ahora puede encargarse de la generación rutinaria de código y la depuración de problemas. Esto te permite dedicar el 70 % de tu energía al análisis de requisitos, las decisiones arquitectónicas y la resolución de problemas empresariales complejos. En teoría, estamos lanzando productos al mismo ritmo, pero ahora llegan a la producción bien pensados y con menos errores y casos extremos que resolver.

Veamos un problema con el que quizá estés familiarizado. Imagina que vas a crear una API REST para el dominio en el que trabajas. Antes de la IA, quizá tuvieras que dedicar horas a crear objetos de dominio, lógica de validación, controladores, repositorios y mucho más. Con la IA, unas buenas técnicas de prompt y una plantilla de cómo son las API REST en tu organización, puedes generar estos componentes en cuestión de minutos.

Con el tiempo que ahorras al no tener que escribir código repetitivo, ahora puedes centrarte en algunas de las decisiones estratégicas que realmente importan. En lugar de limitarte a enviar esa API, puedes de-

dicar más tiempo a pensar en el diseño de la API y en si es intuitiva. ¿Has pensado en cómo va a gestionar esta API un alto rendimiento? ¿Estás aprovechando las características de concurrencia del lenguaje o marco con el que trabajas? ¿Esta API se escalará y fallará correctamente bajo carga? Sí, tu aplicación tiene que funcionar primero, pero estos son los tipos de problemas complejos en los que deberías dedicar tu tiempo.

NOTA

Aquí Dan. Si te preocupa que la IA reduzca las líneas de código que escribes, no te preocupes. Las líneas de código no son realmente una medida significativa de la productividad o el valor. He revisado innumerables currículos a lo largo de los años y nunca he visto a nadie incluir «escribí 10 000 líneas de código» como un logro, ni me impresionaría si lo hiciera. Los logros reales se centran en el impacto: «Ahorré a la empresa un 20 % en nuestra factura de la nube al optimizar nuestra arquitectura de aplicaciones» cuenta una historia mucho más convincente que cualquier recuento de líneas.

El cambio de escribir código a resolver problemas no disminuye tu papel en absoluto, sino que lo eleva. Estás resolviendo problemas más grandes y tomando decisiones que tienen un mayor impacto en el equipo y en las aplicaciones en las que trabajas.

Conclusión

La IA no va a sustituir a los desarrolladores de software, pero los desarrolladores que aprovechen eficazmente la IA tendrán una ventaja significativa sobre los que no lo hagan. Al igual que el telar Jacquard transformó a los tejedores en lugar de eliminarlos, la IA está transformando la forma en que abordamos el desarrollo de software.

La IA funciona mejor como tu compañero de programación en pareja, no como tu sustituto. Tanto si utilizas chatbots independientes para debates sobre arquitectura, asistentes IDE en línea para código repetitivo o entornos agenticos para refactorizaciones complejas, el objetivo es el mismo: amplificar tus habilidades existentes y liberar energía mental para la resolución de problemas de mayor nivel.

Comprender las capacidades y limitaciones de la IA es un componente importante para utilizarla de forma eficaz. La IA destaca en tareas repetitivas, explicación de código, generación de documentación y reconocimiento de patrones. Sin embargo, tiene dificultades con el conocimiento en tiempo real, puede generar información incorrecta, carece de contexto empresarial y puede introducir riesgos de seguridad. Tratar el código generado por IA con el mismo rigor que el código escrito por humanos mediante pruebas exhaustivas y revisiones de código no es solo una buena práctica, es esencial.

El futuro del desarrollo de software cambia tu enfoque de escribir código a resolver problemas. En lugar de dedicar el 70 % de tu tiempo a la implementación de plantillas, puedes dedicar esa energía a decisio-

nes arquitectónicas, análisis de requisitos y lógica empresarial compleja. Esto no disminuye tu papel como desarrollador, sino que lo eleva para trabajar en retos más significativos e impactantes.

A medida que empieces a incorporar la IA en tu flujo de trabajo de desarrollo, recuerda que la ingeniería de prompts es una habilidad que se puede aprender y que mejora drásticamente tus resultados. Sé específico sobre el contexto, proporciona ejemplos, estructura tus solicitudes con claridad y no dudes en iterar tus prompts. Y lo más importante, mantén la propiedad de tu código independientemente de quién o qué lo haya generado.

Los desarrolladores que prosperarán en este futuro mejorado por la IA son aquellos que adopten estas herramientas sin dejar de lado su compromiso con la calidad del código, el aprendizaje continuo y los fundamentos de la ingeniería de software. La IA te da superpoderes, pero tú eres responsable de utilizarlos correctamente.

A lo largo de este libro, hemos compartido las lecciones que hemos aprendido con esfuerzo a lo largo de nuestras carreras. Nos sentimos honrados de compartir nuestras experiencias para ayudaros, la próxima generación de ingenieros de software, a alcanzar vuestros objetivos. Esperamos que hayáis disfrutado del libro tanto como nosotros disfrutamos escribiéndolo.

A menudo nos puedes encontrar en reuniones y conferencias, y esperamos que te tomes un momento para presentarte si nos cruzamos. *Gracias por* tomarte el tiempo de leer este libro. ¡ *Te* lo agradecemos más de lo que imaginas!

Ponerlo en práctica

Para poner en práctica lo que has aprendido se requiere acción, no solo conocimientos. Los siguientes pasos prácticos te ayudarán a integrar las herramientas de IA en tu flujo de trabajo de desarrollo, al tiempo que mantienes la calidad del código y desarrollas habilidades esenciales de ingeniería de prompts:

1. Toma un método existente en tu código base y pide a la IA que explique lo que hace, identifique posibles mejoras y sugiera casos de prueba. Compara su análisis con tu propia comprensión.
2. Practica la ingeniería de prompts creando tres prompts bien estructurados, detallados y específicos del contexto utilizando las técnicas de este capítulo para tareas de codificación comunes como la depuración, la revisión de código o la documentación.
3. Crea una «biblioteca de prompts de IA» personal guardando entre cinco y diez prompts eficaces que hayas creado para tareas de desarrollo comunes, como revisiones de código, generación de documentación y depuración.
4. Elige un asistente de codificación de IA y utilízalo durante una semana en un proyecto personal. Documenta en qué tareas destaca y en cuáles tiene dificultades.
5. Practica las «revisiones de código vibe» haciendo que la IA genere una clase o método sencillo y, a continuación, realiza una revisión exhaustiva del código como si lo hubiera escrito un desarrollador junior. Documenta los problemas que encuentres.
6. Utiliza la IA para ayudar a convertir un pequeño fragmento de código de un lenguaje de programación a otro (como de Python a

Java) y, a continuación, verifica manualmente que la conversión sea correcta y sigue las buenas prácticas.
7. Establece tus directrices de uso de la IA definiendo cuándo utilizarás y cuándo no utilizarás la asistencia de la IA, qué tipos de código requieren un escrutinio adicional y cómo manejarás la información confidencial o privada.

Recursos adicionales

- *Ingeniería de IA*, de Chip Huyen (O'Reilly, 2024)
- *Beyond Vibe Coding*, de Addy Osmani (O'Reilly, 2025)
- *Ingeniería de Prompts para LLMs*, de John Berryman y Albert Ziegler (O'Reilly, 2024)
- «No Silver Bullet», de Fredrick P. Brooks Jr. (Universidad de Carolina del Norte en Chapel Hill)
- *Rebooting AI: Building Artificial Intelligence We Can Trust*, de Gary Marcus (Vintage, 2020)
- *Co-Intelligence: Living and Working with AI*, de Ethan Mollick (Portfolio, 2024)
- *Human Compatible: Artificial Intelligence and the Problem of Control*, de Stuart Russell (Penguin Books, 2020)
- *Artificial Intelligence: A Modern Approach*, de Stuart Russell y Peter Norvig (Pearson, 2021)
- *Deep Learning con Python*, de Francois Chollet (O'Reilly, 2021)
- *Domesticando Silicon Valley*, de Gary Marcus (Tantor Media, Inc., 2025)
- Blog de Simon Willison

- Marcus sobre la IA
- Podcast sobre IA práctica